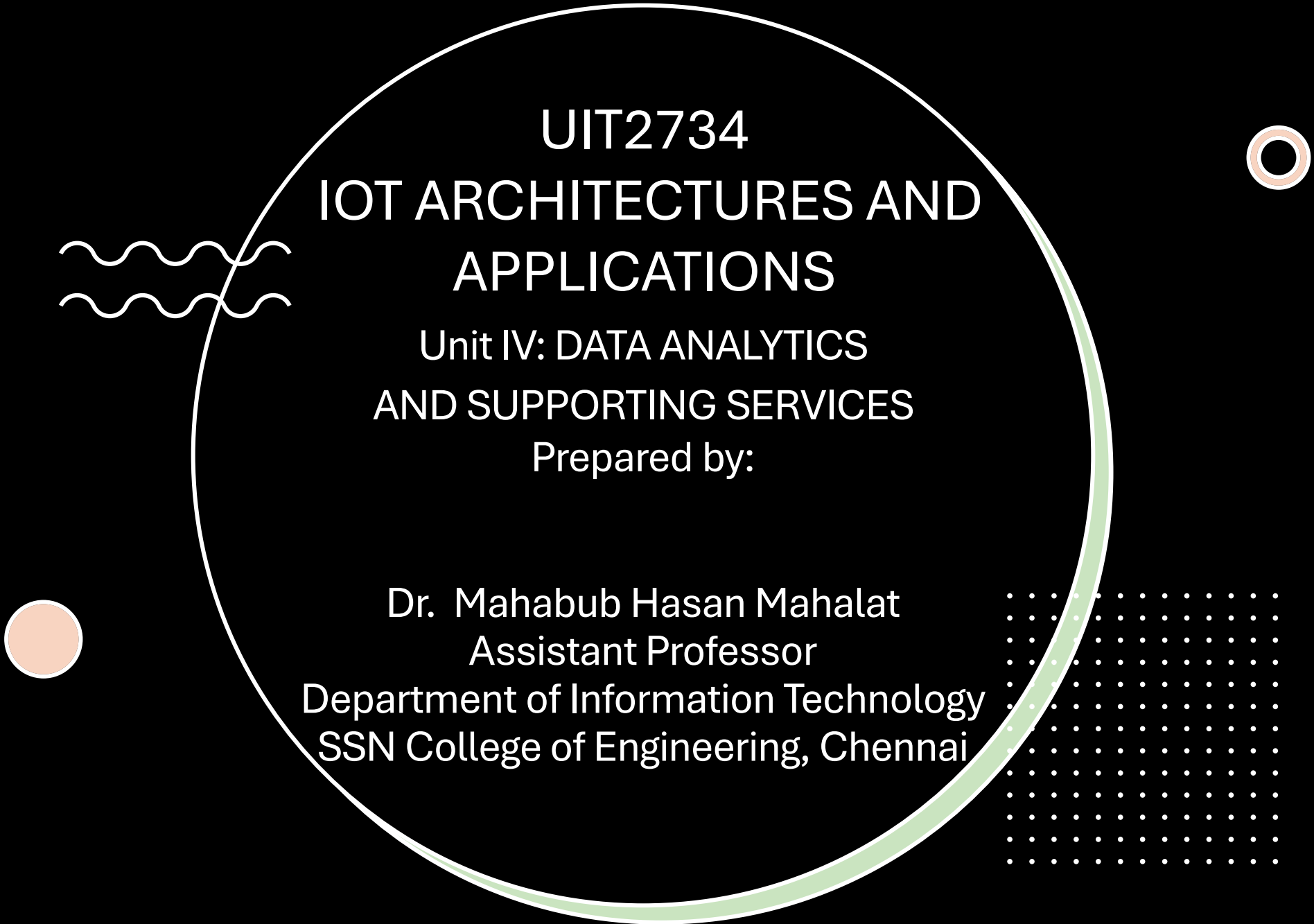




UIT2734

IOT ARCHITECTURES AND APPLICATIONS

Unit IV: DATA ANALYTICS
AND SUPPORTING SERVICES

Prepared by:

Dr. Mahabub Hasan Mahalat
Assistant Professor
Department of Information Technology
SSN College of Engineering, Chennai

Data in the IoT Ecosystem

- IoT generates massive volumes of heterogeneous data from connected devices (sensors, actuators, gateways)
- Data characteristics vary by source:
 - Temperature sensors → numeric, structured
 - Security cameras → video streams, unstructured
 - Log files → semi-structured
- **Two critical data classification axes in IoT :**
 - Format: Structured vs Unstructured
 - State: Data in Motion vs Data at Rest
- Choosing the right data management strategy depends on these classifications
- Proper data architecture is foundational to scalable IoT applications

Structured vs Unstructured Data

Structured Data

- Data organized in a predefined format (rows & columns); easily searchable by relational databases

- Key Characteristics:

- Fixed schema and data types
- Stored in relational databases (SQL)
- Highly organized and easily queryable
- Quantitative in nature

- IoT Examples:

- Sensor readings: temperature, pressure, humidity
- Device timestamps and event logs
- GPS coordinates from fleet tracking systems
- Energy meter readings in smart grids

- Facilitates real-time analytics and machine-to-machine (M2M) communication

Examples of structured data



Pricing data



CRM data



Dates & times



Financial transactions



Customer account data



Medical information

Unstructured Data

- Data without a predefined schema or format; does not fit neatly into tables or relational databases
- Key Characteristics:
 - No fixed schema; heterogeneous formats
 - Difficult to search and analyze using traditional tools
 - Requires advanced techniques: NLP, computer vision, ML
 - Qualitative and high volume
- IoT Examples:
 - Video streams from surveillance cameras
 - Audio recordings from smart devices (voice assistants)
 - Social media data and user feedback
 - Images from medical IoT imaging devices
- **Constitutes ~80–90% of all IoT-generated data**



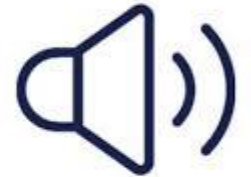
Image



Text



Video



Audio



Email



Social Media



Sensor

Structured vs Unstructured Data – Comparison

Aspect	Structured Data	Unstructured Data
Format	Fixed, predefined schema	No unified structure
Representation	Rows and columns (tabular)	Text, images, video, audio
Storage	Relational databases (RDBMS)	NoSQL, data lakes, object storage
Query Language	SQL	No standard query language
Searchability	Easily searchable	Difficult to search
Storage Space	Less storage required	More storage required
Security	Enforced privileges – easier to protect	More complex to secure
Processing	Efficient for defined patterns	Requires advanced analytics (AI/ML)
Flexibility	Rigid – schema changes are difficult	Flexible – adapts to evolving data types

IoT-Specific Observations

Observation	Implication
Structured data holds a minor share of total IoT-generated data	Most IoT data is unstructured
Structured and unstructured data coexist in IoT systems	Systems must handle both types
Industrial IoT often combines both approaches	Hybrid data management is essential

Semi-Structured Data

A third category exists between structured and unstructured data:

Aspect	Description
Definition	Data that has some organizational properties but lacks a rigid schema
Examples	JSON, XML, system logs
IoT Relevance	Common in IoT device communication and APIs

Data in Motion vs Data at Rest

Data in Motion

- Data actively moving through a network — being transmitted from one point to another in real time
- Also called: Streaming data / Data in transit
- Key Characteristics:
 - Time-sensitive; requires low-latency processing
 - Flows between sensors, gateways, cloud, and applications
 - Processed using stream processing frameworks
 - Vulnerable to interception; requires encryption in transit (TLS)
- IoT Examples :
 - Real-time patient vitals from wearables to hospital servers
 - Live telemetry from autonomous vehicles
 - Sensor data streams from industrial machines (SCADA)
- Technologies: MQTT, AMQP, Apache Kafka, Flink

Data at Rest

- Data stored persistently in a storage medium — not actively moving through a network
- Also called: Stored data / Static data
- Key Characteristics:
 - Stored in databases, data lakes, or file systems
 - Used for historical analysis, batch processing, and reporting
 - Requires encryption at rest (AES-256) for security
 - Subject to data retention and compliance policies
- IoT Examples :
 - Historical sensor logs stored in cloud databases
 - Archived machine maintenance records
 - Stored images and video clips in smart home systems
- Technologies: Amazon S3, Hadoop HDFS, InfluxDB, Cassandra

Data in Motion vs Data at Rest – Side by Side

Data in Motion

- Active data transmission over networks
- Real-time or near-real-time processing
- High velocity; low latency required
- Protocols: MQTT, HTTP/2, WebSocket
- Tools: Apache Kafka, AWS Kinesis
- Security: TLS/SSL encryption in transit
- Use case: Live dashboard alerts

Data at Rest

- Stored data in databases / file systems
- Batch processing and historical queries
- Lower velocity; high durability required
- Formats: JSON, CSV, Parquet, Avro
- Tools: Hadoop, S3, InfluxDB, MySQL
- Security: AES-256 encryption at rest
- Use case: Monthly energy consumption reports

Data in Motion vs Data at Rest – Comparison

Aspect	Data in Motion	Data at Rest
State	Actively transmitted	Stored persistently
Processing	Stream processing	Batch processing
Latency	Low latency (ms–sec)	Higher latency acceptable
Tools	Kafka, MQTT, Flink	Hadoop, S3, HDFS, SQL DBs
Security Risk	Interception, MITM attacks	Unauthorized access, theft
Security Method	TLS/SSL encryption	AES-256 encryption
IoT Example	Live vehicle GPS tracking	Archived trip history logs
Priority	Speed & real-time insight	Durability & compliance

IoT Data Pipeline – Putting it Together

- **A complete IoT system handles all four data types in a unified pipeline:**
 - STEP 1 – Sensing: Structured (sensor readings) & Unstructured (images/audio) data generated
 - STEP 2 – Transmission: Data in Motion flows via MQTT/HTTP from device → gateway → cloud
 - STEP 3 – Processing: Stream analytics (Kafka/Flink) for real-time; ML models for unstructured
 - STEP 4 – Storage: Data at Rest stored in time-series DB (InfluxDB) or data lake (S3/HDFS)
 - STEP 5 – Analytics & Visualization: Dashboards, batch reports, predictive maintenance
- Reference Architecture :
 - IoT Platform → Cloud Storage → Big Data Processing → Application Services
- Security must be applied at every step: in transit and at rest

Real-World IoT Use Cases

IoT Domain	Data Type	Motion/Rest	Example
Smart Healthcare	Structured	In Motion	Real-time ECG monitoring via wearables
Smart Healthcare	Unstructured	At Rest	Archived MRI scan images
Smart Manufacturing	Structured	In Motion	Live machine vibration telemetry
Smart Grid	Structured	At Rest	Historical energy consumption logs
Smart City	Unstructured	In Motion	Traffic camera live video feeds
Agriculture IoT	Structured	At Rest	Stored soil moisture history
Autonomous Vehicles	Unstructured	In Motion	LiDAR point cloud streaming

Dr. Mahabub Hasan Mahalat

Big Data Technologies in IoT

Role of Machine Learning · NoSQL Databases
Hadoop Ecosystem · Apache Kafka · Apache Spark

Why Big Data Analytics Matters in IoT

50+ Billion

IoT devices by 2030

175 ZB

Global data by 2025

~90%

IoT data unstructured

Milliseconds

Required decision latency

IoT Data Challenges :

- **Volume:** petabytes of sensor data daily
- **Velocity:** real-time stream processing required
- **Variety:** structured, unstructured, semi-structured
- **Veracity:** noisy, incomplete sensor readings
- **Value:** extract actionable insight from raw data

Big Data: Extremely large datasets characterized by the 3Vs — Volume, Velocity, and Variety, requiring advanced technologies for efficient storage, processing, and analysis.

Big Data Solutions in IoT:

- **NoSQL DBs** for scalable heterogeneous storage
- **Hadoop** for batch processing of large datasets
- **Kafka** for high-throughput stream ingestion
- **Spark** for in-memory, real-time analytics
- **ML** for pattern recognition & predictive models

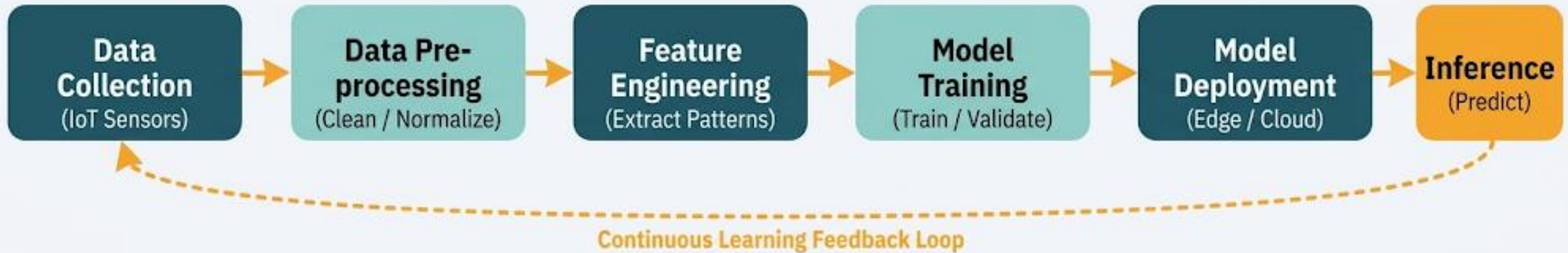
Role of Machine Learning in IoT

Definition & Significance

- **Machine Learning (ML):** A subset of AI where systems learn from data without being explicitly programmed
- In IoT, ML enables devices and platforms to:
 - Detect anomalies in sensor data automatically (e.g., unusual vibration in a motor)
 - Predict failures before they happen — Predictive Maintenance
 - Optimize energy consumption dynamically in smart buildings
 - Recognize patterns in images, voice, and time-series data
 - Enable autonomous decision-making without human intervention
- **Why ML is Critical in IoT :**
 - Rule-based systems fail at scale — ML generalizes from millions of data points
 - IoT data is too large and fast for manual analysis
 - Edge ML (TinyML) enables low-latency inference directly on devices

ML Workflow & Types in IoT Systems

ML Workflow in IoT Systems



ML Techniques Used in IoT



Machine Learning Applications in IoT

Predictive Maintenance

- Industrial sensors → detect vibration anomalies
- LSTM networks on time-series data
- Prevent costly unplanned downtime

Smart Energy Management

- Regression models for energy forecasting
- RL agents control HVAC automatically
- Reduce energy costs by up to 30%
- Used in smart grid IoT systems

Healthcare Monitoring

- CNN + LSTM on wearable sensor data
- Detect cardiac arrhythmia in real-time
- Alert physicians before critical event

Smart Transportation

- Object detection for autonomous vehicles
- Traffic flow prediction (LSTM)
- Route optimization via RL agents
- Real-time V2X communication ML

Agriculture (AgrIoT)

- Soil + weather sensors → crop yield prediction
- Image-based plant disease detection
- Automated irrigation scheduling

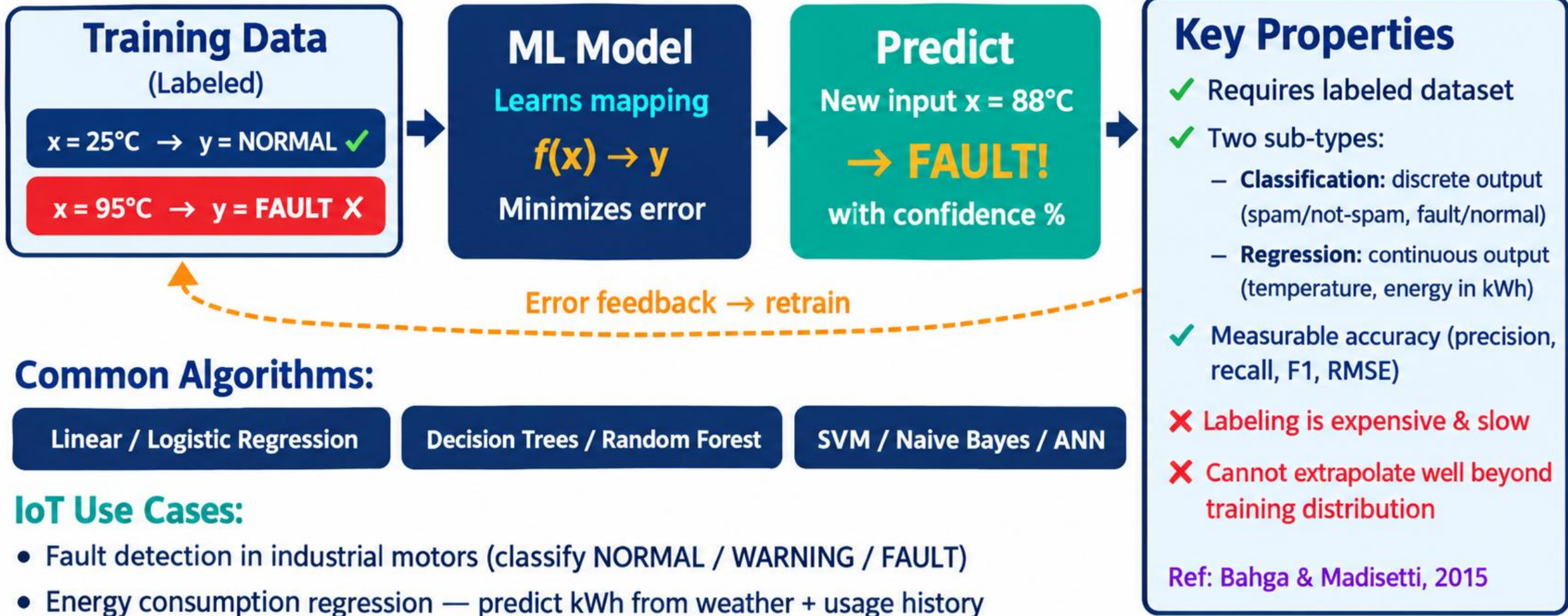
Edge AI / TinyML

- Deploy ML on microcontrollers (MCU)
- TensorFlow Lite, Edge Impulse
- Sub-millisecond inference on-device
- No cloud needed; ultra-low power

Supervised vs Unsupervised vs Reinforcement vs Deep Learning – Differences

Aspect	Supervised	Unsupervised	Reinforcement	Deep Learning
Training Data	Labeled (input + output pairs)	Unlabeled (raw data only)	No labels; uses rewards/penalties	Labeled or unlabeled (large volumes)
Learning Approach	Learn from correct answers	Find hidden patterns/groups	Trial & error with reward signal	Learn features automatically via layers
Human Involvement	High (manual data labeling)	Low (no labeling needed)	Medium (design reward function)	Low after initial dataset prep
Output Type	Class label or numeric value	Clusters or reduced dimensions	Action/policy for each state	Any: class, value, image, text
Key Algorithms	SVM, Random Forest, Linear Reg.	K-Means, DBSCAN, PCA	Q-Learning, DQN, PPO, A3C	CNN, LSTM, Transformer, Autoencoder
IoT Application	Fault classification, forecasting	Device grouping, anomaly detect	HVAC control, robot navigation	Image defect detect, time-series predict
Data Volume Needed	Moderate (thousands)	Low to moderate	Many simulation episodes	Very large (millions of samples)
Interpretability	High (decision trees, coefficients)	Medium (cluster analysis)	Medium (policy readable)	Low (black-box deep network)
Compute Cost	Low to moderate	Low to moderate	High (many simulation steps)	Very high (GPU/TPU required)

① Supervised Learning



② Unsupervised Learning



Common Algorithms:

K-Means Clustering

DBSCAN / Hierarchical

PCA / Autoencoders

IoT Use Cases:

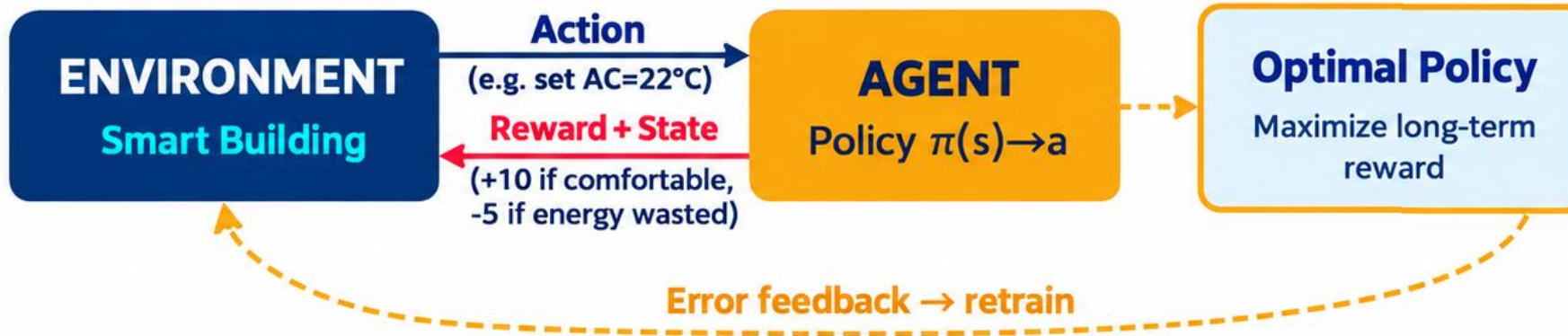
- Anomaly detection — find unusual patterns in sensor data without prior labeling
- Group IoT devices into behavior profiles (active / idle / faulty) automatically

Key Properties

- ✓ No labeled data needed
- ✓ Two main tasks:
 - Clustering: group similar data points together
 - Dimensionality Reduction: compress features (PCA)
- ✓ Discovers hidden structure
- ✓ Great for exploration
- ✗ Results hard to interpret
- ✗ No ground-truth to validate
- ✗ Number of clusters (K) must be pre-specified

Ref. James et al., 2017

③ Reinforcement Learning



Common Algorithms:

Q-Learning / DQN

Policy Gradient (PPO)

Actor-Critic (A3C)

IoT Use Cases:

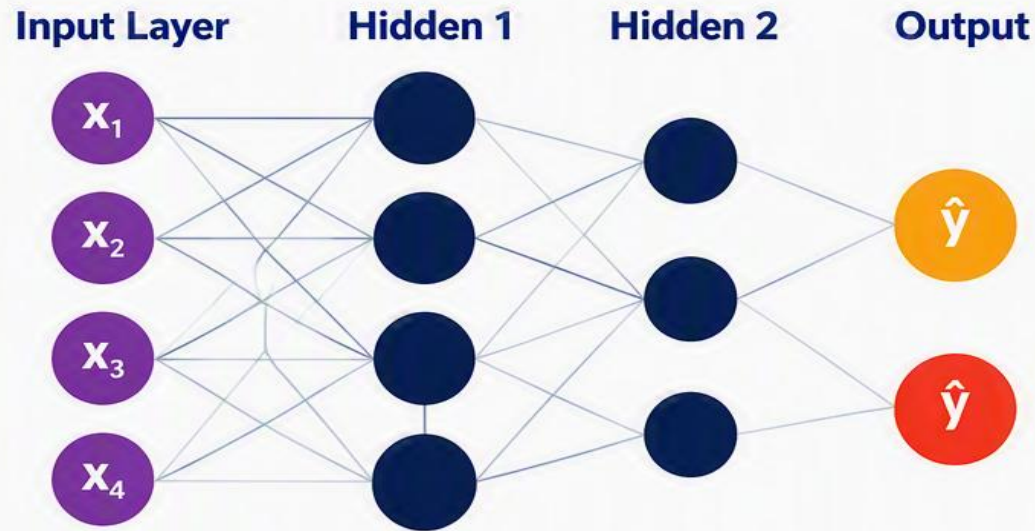
- Smart HVAC control — agent learns optimal temperature set-points to save energy
- Autonomous robot navigation in warehouses / factories without pre-programmed rules

Key Properties

- ✓ No labeled data needed
- ✓ Learns via trial & error
- ✓ Handles sequential decisions
- ✓ Adapts to dynamic environments
- ✓ 4 core components:
 - Agent (decision maker)
 - Environment (world)
 - Reward (feedback signal)
 - Policy (strategy π)
- ✗ Slow to train (many episodes)
- ✗ Reward function design is hard
- ✗ Can find unexpected shortcuts

Ref: Bahga & Madisetti, 2015

④ Deep Learning (DL)



DL Architectures for IoT:

CNN (Image/Video)
Camera-based defect detection

LSTM (Time-Series)
Predictive maintenance, ECG

Autoencoder (Anomaly)
Unsupervised anomaly detection

TinyML / Edge AI
Deploy on MCUs (Arduino, Pi)

Key Properties

- ✓ Automatic feature learning
- ✓ Handles raw unstructured data (images, audio)
- ✓ State-of-the-art accuracy
- ✓ Scalable to big data
- ✓ Transfer learning: reuse pre-trained models
- ✗ Needs huge datasets
- ✗ Black box — low interpretability
- ✗ High compute cost
- ✗ GPU required

Ref: Baheza & Madiseti,
Hanes et al., 2017

Common Tools:

TensorFlow / Keras

PyTorch

TensorFlow Lite

IoT Use Cases:

- LSTM-based predictive maintenance — predict machine failure 48h in advance from vibration time-series

NoSQL Databases for IoT

NoSQL Databases – Definition & Need in IoT

- NoSQL ("Not Only SQL"): Non-relational databases designed for scalability, flexibility, and high performance
- **Why SQL is Insufficient for IoT:**
 - **Fixed schema** fails with heterogeneous sensor data formats
 - Vertical scaling is expensive; NoSQL **scales horizontally** (add nodes)
 - SQL struggles with millions of writes/sec from concurrent IoT devices
 - ACID transactions are not always required for sensor data workloads
- **NoSQL Advantages for IoT :**
 - **Schema-less:** device metadata can evolve without schema migration
 - **Horizontal scaling:** distribute across commodity servers globally
 - Eventual consistency: acceptable for sensor telemetry pipelines
 - CAP Theorem: NoSQL systems balance Consistency, Availability, Partition Tolerance
- Examples: MongoDB (document), Cassandra (column), Redis (key-value), Neo4j (graph)

NoSQL Database Types & IoT Use Cases

Type	Description	Best For	Examples
Document Stores	Stores JSON-like documents; flexible schema	Semi-structured sensor data	MongoDB, CouchDB
Key-Value Stores	Simple key-value pairs	Device state, caching	Redis, DynamoDB
Column-Family Stores	Stores data in columns rather than rows	Time-series data, analytics	Cassandra, HBase
Time-Series Databases (TSDB)	Optimized for time-stamped sensor data	Continuous sensor data streams	InfluxDB, GridDB

A hybrid approach **combines the transactional consistency of SQL with the horizontal scalability of NoSQL** – outperforming solutions based exclusively on either approach for high-volume, heterogeneous IoT data

NoSQL Database Types – Detailed Comparison

Type	Examples	Data Model	Consistency	Best IoT Use Case
Key-Value	Redis, DynamoDB	Key - Value pairs	Strong / Eventual	Device state cache, session tokens
Document	MongoDB, CouchDB	JSON/BSON documents	Eventual	Device metadata, patient health records
Column-Family	Cassandra, HBase	Rows + dynamic columns	Tunable	Time-series sensor streams (writes)
Time-Series	InfluxDB, TimescaleDB	Timestamped metrics	Strong	Industrial IoT, monitoring dashboards

Apache Hadoop – Overview & IoT Relevance

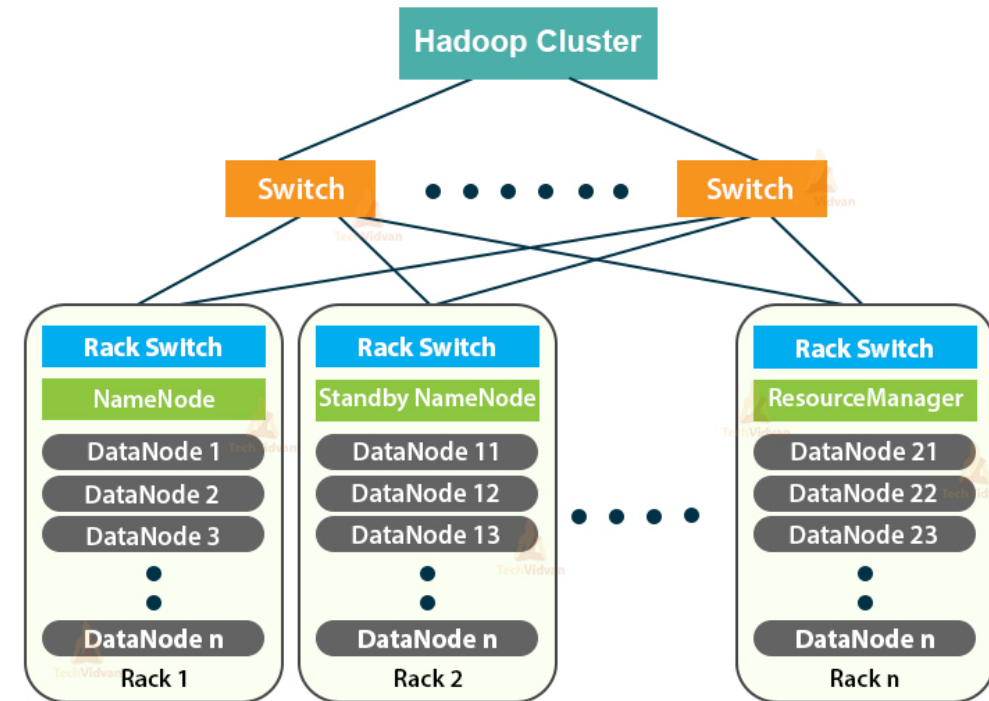
- The Hadoop ecosystem provides a framework for **distributed storage and processing** of massive IoT datasets across clusters of computers

- **Four Core Components:**

- Hadoop Common: Shared utilities and libraries used by all modules
- HDFS: Hadoop Distributed File System — stores data across multiple nodes
- MapReduce: Parallel data processing programming model
- YARN: Yet Another Resource Negotiator — cluster resource management

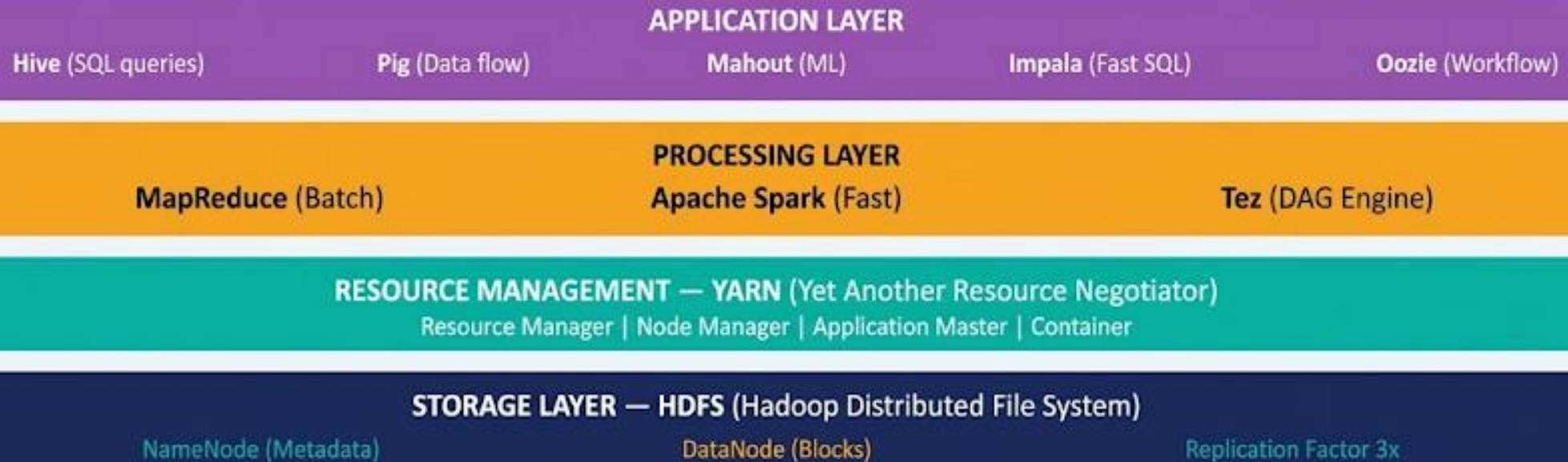
- **Why Hadoop for IoT :**

- Store and process historical sensor data from thousands of devices
- Fault tolerance: data replicated 3× across nodes; survives hardware failure
- Cost-effective: runs on commodity hardware, not expensive servers
- Batch analytics: monthly energy reports, predictive maintenance histories



Hadoop Ecosystem Architecture

Hadoop Ecosystem Architecture

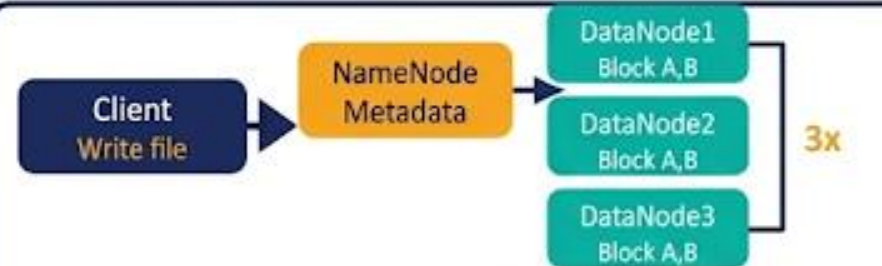


MapReduce Process



IoT: Process millions of sensor records in parallel
Write Once, Read Many (WORM) — fault tolerant

Dr. Mahabub Hasan Mahalat



Block size: 128MB default | Automatic failover

Hadoop Ecosystem Architecture

1. Application Layer

Includes Hive (SQL-like queries), Pig (data flow scripting), Mahout (machine learning), Impala (fast interactive SQL), Sqoop/Flume for ingestion, and Oozie for workflow scheduling.

2. Processing Layer

Powers computing via MapReduce (batch processing), Apache Spark (in-memory engine with MLlib, SQL and streaming), and Tez (DAG execution engine) for low-latency pipelines.

3. Resource Management (YARN)

Cluster management using ResourceManager (global scheduling), NodeManager (per-node monitoring), ApplicationMaster (per-job negotiation), and Containers for CPU/memory allocation.

4. Storage Layer (HDFS)

Distributed store with NameNode (metadata and namespace) and DataNodes holding 128 MB blocks, with 3x replication, rack awareness, and automatic failover for fault tolerance.

HDFS – Hadoop Distributed File System

How HDFS Works:

- Files split into fixed-size blocks (128 MB default)
- Each block replicated 3× across different DataNodes
- NameNode: stores metadata (file names, locations, block map)
- DataNode: stores actual data blocks
- Secondary NameNode: checkpoints namespace; not a failover
- HDFS Federation: multiple NameNodes for scalability
- High Availability: Active/Standby NameNode via ZooKeeper

HDFS Properties for IoT:

- Write once, read many times — perfect for sensor logs you never edit
- Reads whole files start-to-end (fast), not tiny random bits (slow)
- Copies are kept on different server racks, so one failure loses nothing
- Work is sent to the machine holding the data, instead of moving the data
- Common commands:
 - `hdfs dfs -put sensor_logs/ /iot/data/` (upload)
 - `hdfs dfs -ls /iot/data/` (list files)

MapReduce – Parallel Processing in Hadoop

- MapReduce: Programming model to process large datasets in parallel across cluster by dividing the task into Map and Reduce phases

➤ Phase 1 – MAP:

- Input: raw data split into key-value pairs by InputFormat
- Mapper function: processes each record, emits intermediate (**key, value**) pairs
- Example: Mapper reads each sensor line → emits (device_id, temperature)

➤ Phase 2 – SHUFFLE & SORT:

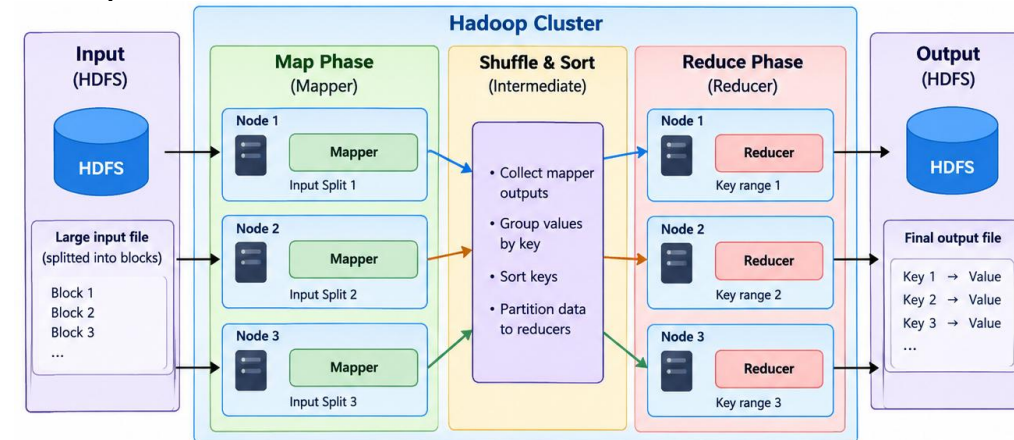
- Framework groups all values for the same key together
- Data transferred across network to Reducer nodes

➤ Phase 3 – REDUCE:

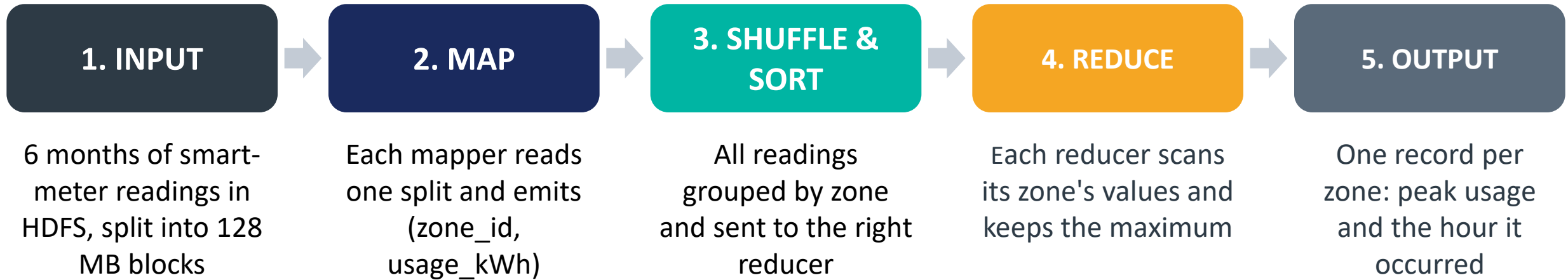
- Reducer function: aggregates/summarizes grouped values
- Example: Reducer sums temperatures per device → (device_id, avg_temp)
- IoT Use Case: Process 6 months of smart meter data in parallel — find peak usage per zone

- **Limitation: High latency (disk I/O heavy) — not suitable for real-time. Apache Spark solves this.**

Dr. Mahabub Hasan Mahalat



MapReduce Flow — IoT Smart Meter Use Case



Worked example

Raw record: 2024-03-01 08:15, meter_77, Zone A, 4.2 kWh → Map emits: (Zone A, 4.2)

Shuffle groups: (Zone A, [4.2, 6.8, 5.1]) → Reduce max(): 6.8 → Output: (Zone A, peak 6.8 kWh)

Hadoop Ecosystem Tools – Hive, HBase & Pig

Apache Hive

- SQL-like query language (HiveQL) on top of Hadoop
- Converts SQL to MapReduce/Tez/Spark jobs
- Schema-on-read: define structure at query time
- IoT Use: Query months of archived sensor data
- Example: `SELECT device, AVG(temp) FROM logs GROUP BY device`

Apache HBase

- Distributed, column-oriented NoSQL DB on HDFS
- Modeled after Google Bigtable
- Random read/write access (unlike HDFS)
- Row key = device_id + timestamp (time-series)
- IoT Use: Real-time lookups in large sensor tables
- Supports millions of columns per row

Apache Pig

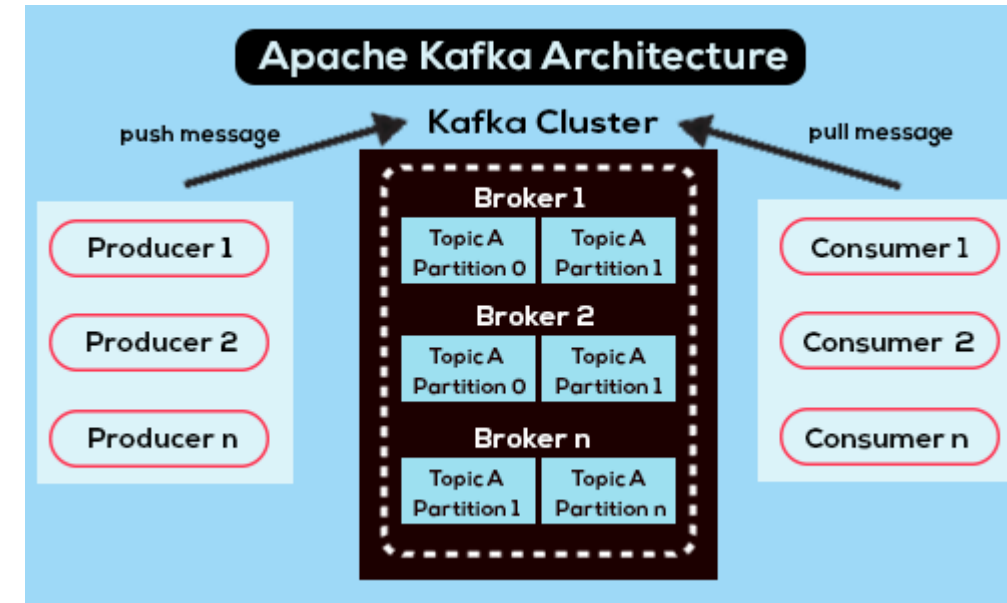
- High-level scripting language (Pig Latin) for data flows
- Automatically parallelizes across MapReduce
- Self-optimizing: no need to manually tune MR
- IoT Use: Data cleansing & ETL pipelines
- Example: `FILTER sensor BY temp > 100`
- Procedural, easier than writing raw MR code

Apache Kafka

Distributed Publish-Subscribe Messaging for IoT Data Streams

Apache Kafka – Overview & Core Concepts

- Apache Kafka: Distributed, fault-tolerant, high-throughput publish-subscribe messaging system originally developed at LinkedIn (2011); open-sourced by Apache
- **Core Abstractions:**
 - **Producer:** application that publishes messages to Kafka topics (e.g., IoT sensor)
 - **Consumer:** application that reads messages from topics (e.g., Spark, dashboard)
 - **Topic:** named log/channel where messages are stored and categorized
 - **Partition:** topics split into ordered, immutable partitions for parallelism
 - **Offset:** unique sequential ID of each message within a partition
 - **Broker:** Kafka server node that stores and serves messages
 - **Consumer Group:** set of consumers that cooperatively read from a topic



- **Why Kafka for IoT :**
 - Handles millions of messages/second from thousands of IoT devices
 - Durable: messages persisted to disk and replicated for fault tolerance
 - Decouples data producers from consumers — flexible, scalable architecture

Apache Kafka Architecture in IoT

PRODUCERS



IoT Sensors

Temp, Humidity, CO2



Wearable Devices

Heart rate, SpO2



Vehicles / GPS

Location, Speed



Smart Meters

Energy consumption



Edge Gateway

Aggregated data

Protocol: MQTT/HTTP

KAFKA BROKER CLUSTER

Zookeeper (Cluster Coordination & Leader Election)

TOPICS & PARTITIONS

Topic: sensor-readings

Partition 0

Offset 0→N

Partition 1

Offset 0→N

Partition 2

Offset 0→N

Replica

Topic: alerts

Partition 0

Offset 0→N

Partition 1

Offset 0→N

Partition 2

Offset 0→N

Topic: video-streams

High-throughput unstructured data

Key Features:

Retention: configurable | Throughput: millions msg/sec | Fault-tolerant

CONSUMERS



Apache Spark

Stream processing



ML Models

Real-time inference



Cloud DB

InfluxDB / Cassandra



Dashboards

Grafana / Kibana



Alert Systems

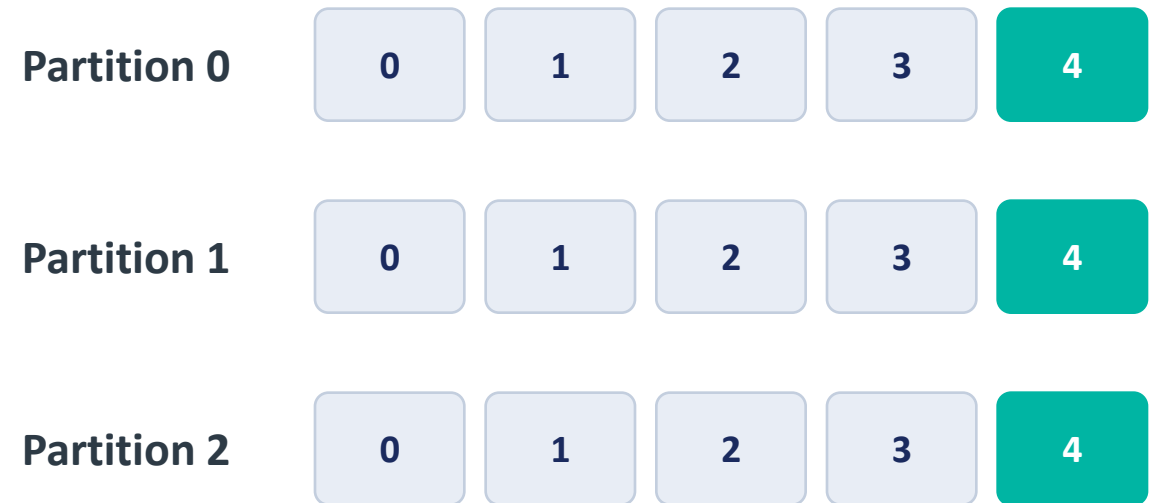
SMS / Email / Push

Kafka Topics & Partitions

Topics & Partitions:

- A Topic is a log of messages; analogous to a database table
- Each topic split into N partitions for parallel processing
- Partitions distributed across brokers in the cluster
- Each partition is an ordered, immutable sequence of messages
- Offset: unique index per message in partition (0, 1, 2...)
- Consumers track their own offset independently
- Replication factor: each partition replicated to R brokers
- Leader partition handles reads/writes; replicas are passive

Topic: `iot/temperature` (3 partitions)



offsets: 0, 1, 2 ... newest appended at the end

- Each partition is an ordered, append-only log. A message is identified by its partition + offset.

Retention: How Long Kafka Keeps a Message

What retention means

- **Retention** = the rule that decides when Kafka deletes old messages.
- **Reading does not delete it:** a message stays until its time or size limit is reached.
- **Like a CCTV recorder:** it holds the last 7 days, then overwrites the oldest.
- **That is why consumers can replay** old data — until it expires.

One partition's log, with a 7-day retention window:



Older than 7 days - > deleted kept — any consumer can still read or replay these

Three ways to set the rule

- **Time-based** (default 7 days) — delete anything older than N days.
- **Size-based** — keep only the last N GB in each partition.
- **Compaction** — keep only the newest message per key.
 - **IoT example:**
 - iot/temperature — raw readings, keep 24 hours
 - iot/alerts — breach events, keep 30 days
 - iot/device-state — compacted: latest value per device kept

Remember: retention is about how long data sits on disk — not about who has read it. Pick it from how far back you might need to replay.

Apache Spark

Apache Spark – Overview & Core Concepts

- Recap: MapReduce writes every intermediate result to disk — slow when a job passes over the same data again and again.
- **What is Spark?** One open-source engine that runs batch, SQL, streaming and ML jobs on a cluster (UC Berkeley, 2009).
- **Key idea — keep the data in memory (RAM)**
 - Spark reuses cached data instead of re-reading it from disk → up to 100× faster for repeated (iterative) work
- **Core building block — RDD (Resilient Distributed Dataset)**
 - **Distributed:** data is split into partitions spread across the cluster nodes
 - **Resilient:** if a node fails, Spark rebuilds only the lost partition from its recipe of steps (DAG lineage)
 - **Read-only and lazy:** nothing runs until you ask for a result (an action)
- Today we mostly write **DataFrames** — RDDs shown as a table with named columns, queried like SQL.
- Runs on: YARN, Mesos, Kubernetes, standalone or local mode | Written in: Python (PySpark), Scala, Java, R, SQL

Apache Spark – Overview & Core Concepts...

- Spark provides **in-memory, distributed processing** for both batch and streaming IoT data.

Feature	Description
Speed	In-memory processing up to 100x faster than Hadoop MapReduce
Streaming (Spark Streaming)	Processes real-time data streams from Kafka
SQL	Query structured data using Spark SQL
MLlib	Built-in machine learning library for IoT analytics
Integration	Reads from Kafka, processes, and writes to Cassandra/HDFS

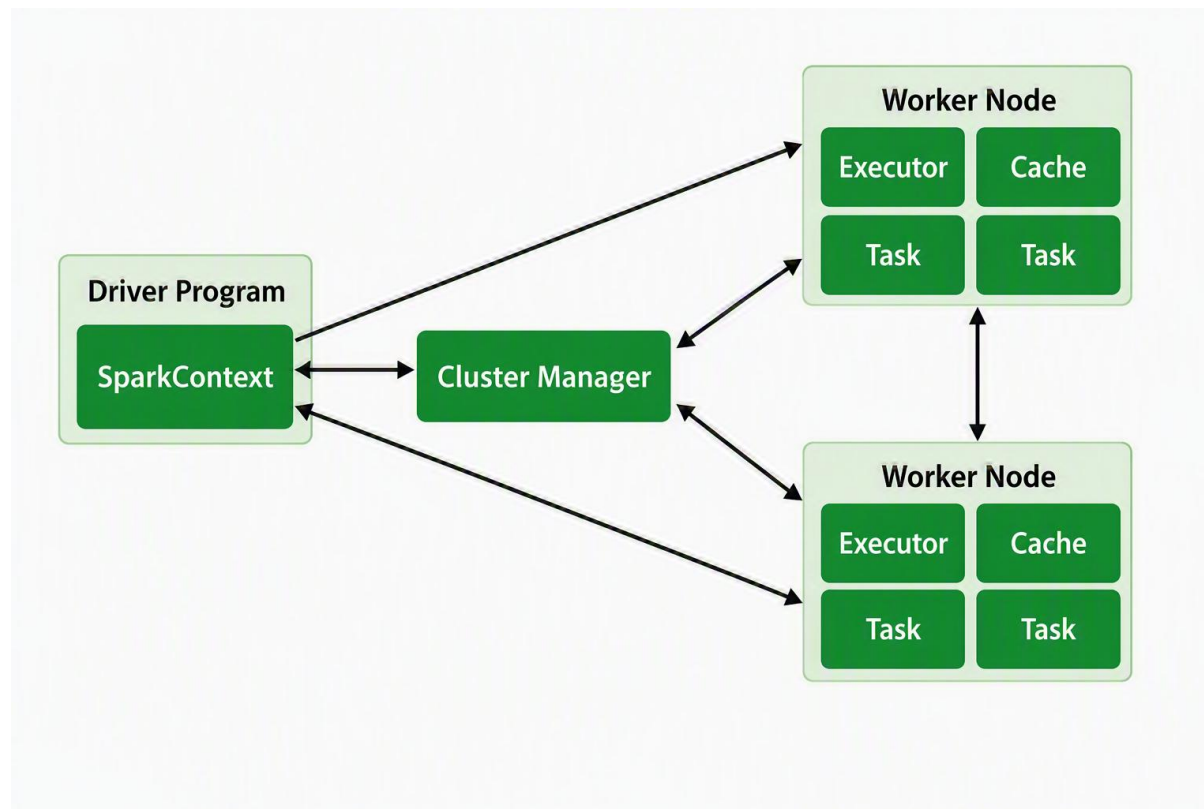
Apache Spark Architecture

Who does what

- **Driver Program** – runs your main(); plans the job
- **SparkContext** – the driver's connection to the cluster
- **Cluster Manager** – grants resources (YARN, Mesos, K8s, Standalone)
- **Worker Node** – a machine in the cluster that does the work
- **Executor** – process on a worker; runs tasks, holds the Cache
- **Task** – one operation on one data partition

How a job runs (follow the arrows)

1. **Driver starts** – your program creates a SparkContext, which asks the Cluster Manager for resources.
2. **Executors launch** – the Cluster Manager starts an Executor process on each Worker Node and reports back to the driver.
3. **Tasks are sent** – the driver splits the job into stages and tasks (one per partition) and ships them to the executors.
4. **Work and results** – executors run tasks, cache hot partitions in RAM, shuffle data between workers, and send results back to the driver.



Spark Built-in Libraries — Unified Engine

- **Apache Spark** provides a **unified analytics engine** with built-in libraries.
 - **Spark SQL** – Structured data processing using SQL/DataFrames.
 - **MLlib** – Machine learning algorithms and pipelines.
 - **GraphX** – Graph processing and graph analytics.
 - **Structured Streaming** – Real-time/stream data processing.
 - **PySpark** – Python API for Spark-based data analytics.
 - All libraries run on the **same Spark engine**, enabling easy integration and data sharing.

Spark MLlib & Spark SQL for IoT Analytics

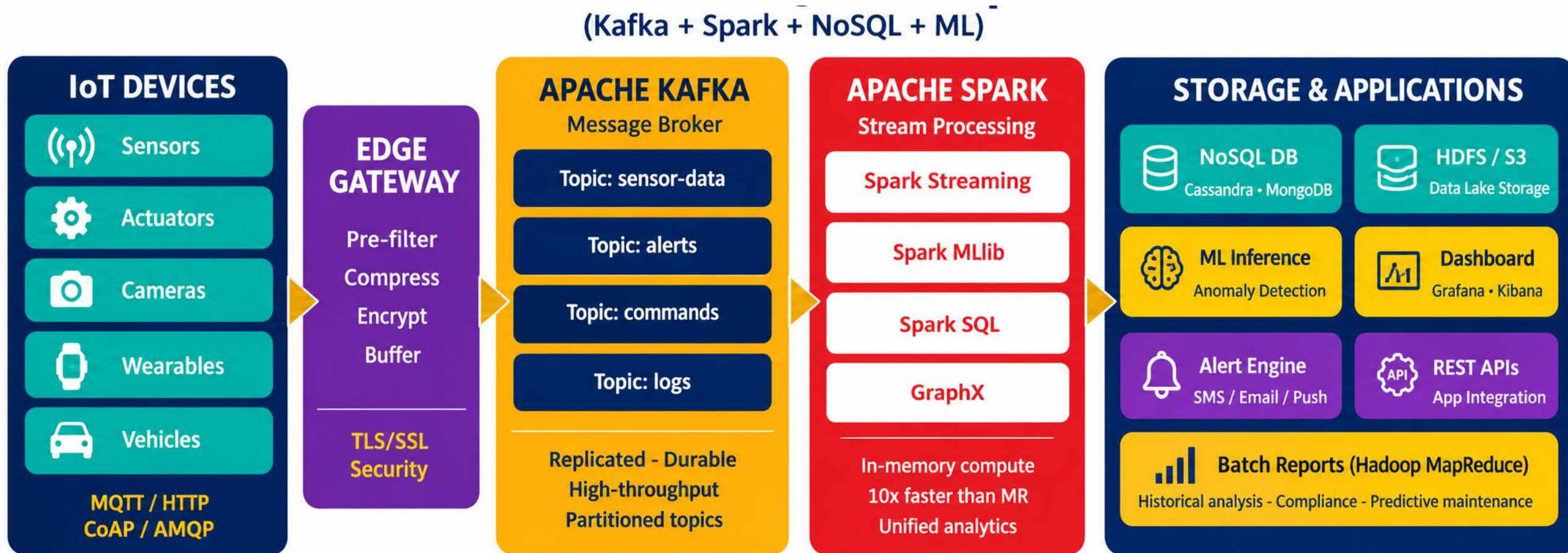
Spark MLlib

- Distributed ML library built into Spark
- Algorithms: Regression, Classification, Clustering
- Random Forest, Gradient Boosted Trees
- K-Means, PCA, ALS (recommender systems)
- Pipelines API: chain transformers + estimators
- Scales to petabytes of training data
- IoT: Train anomaly detection model on 1 year of sensor data
- Ref: Bahga & Madisetti, 2015

Spark SQL

- SQL queries on structured IoT data via DataFrames
- Catalyst Optimizer: auto-optimizes query plan
- Tungsten Engine: CPU/memory efficiency
- Read from: Parquet, ORC, JSON, Avro, JDBC
- Write to: HDFS, S3, Cassandra, RDBMS
- IoT: "SELECT device, MAX(temp) FROM sensors WHERE ts > NOW()-1h GROUP BY device"
- Unified: SQL + streaming + ML in one pipeline

Integrated IoT Big Data Pipeline – Kafka + Spark + NoSQL + ML



Kafka vs Spark vs Hadoop – Technology Comparison

Aspect	Hadoop	Kafka	Spark
What is it?	Storage + Batch Processing	Real-Time Data Streaming	Fast Data Processing Engine
Primary Role	Store & process massive data	Ingest & stream data in real-time	Process data in memory (fast)
Processing	Batch (disk-based)	None (ingestion only)	Batch + Stream + ML
Speed	Slow (disk)	Very Fast (streaming)	Very Fast (in-memory)
Storage	Yes (HDFS)	Yes (Topics)	No (uses HDFS/S3)
Latency	High (minutes–hours)	Very Low (milliseconds)	Low (sub-seconds)
Real-Time?	No	Yes	Yes
Machine Learning	Limited	No	Yes (MLlib)
Best For	Historical batch analytics	Real-time data ingestion	Real-time analytics + ML

IoT Analytics & Cloud Platforms

Edge Streaming Analytics – Concepts & Why It Matters

- **Edge Streaming Analytics:** Processing IoT data streams in real-time at or near the source (edge/fog layer) rather than sending all data to the cloud first
- **Core Problem with Cloud-Only Analytics:**
 - **Bandwidth cost:** streaming raw video/sensor data to cloud is expensive
 - **Latency:** cloud round-trip adds 50–500ms; unacceptable for safety-critical control
 - **Connectivity:** cloud processing fails when network is unavailable
 - **Privacy:** sending all raw data to cloud raises compliance concerns (GDPR)
- **Edge Streaming Analytics Solves This :**
 - **Filter at source:** send only meaningful events, not raw 1000 readings/sec
 - **Act locally:** trigger actuator in <10ms without cloud involvement
 - **Reduce bandwidth:** compress/aggregate before cloud upload (100× reduction)
 - **Resilience:** continue operating during internet outages

Process where data lives; send only insights to the cloud

Edge Analytics vs Cloud Analytics – Comparison

Aspect	Edge Analytics	Cloud Analytics
Processing Location	On-device / Gateway / Fog node	Centralised cloud data centres
Latency	Sub-millisecond to <100ms	100ms to several seconds
Bandwidth	Minimal (only insights sent up)	High (raw data streams)
Availability	Works offline (disconnected)	Requires internet connectivity
Compute Power	Limited (MCU/SBC/GPU edge)	Unlimited (elastic cloud)
Storage	Local buffer (temporary)	Long-term historical storage
ML Inference	TinyML, TF Lite, ONNX	Full deep learning models
Cost	Hardware capex	Pay-per-use opex (ongoing)
Use Cases	Safety-critical, real-time control, privacy	Historical analysis, model training, BI
Examples	Raspberry Pi, AWS Greengrass, Azure IoT Edge Dr. Mahabub Hasan Mahalat	AWS IoT Core, Azure IoT Hub, GCP IoT

Network Analytics for IoT – Overview

- Network analytics is the process of collecting, monitoring, and analyzing network data to optimize performance, enhance security, and improve overall efficiency in IoT deployments.
- **Why Network Analytics is Critical in IoT:**
 - IoT networks are heterogeneous: WiFi, Zigbee, LoRa, 5G, Ethernet — all co-exist
 - Security threats: IoT devices are prime DDoS botnet targets
 - Performance: latency spikes disrupt real-time control systems
 - Scale: smart city networks may have millions of nodes to monitor
- **Five Key Dimensions of Network Analytics:**
 - **Traffic Analysis:** packet flow monitoring, anomaly detection, protocol inspection
 - **Topology Mapping:** auto-discover and visualise all device connections
 - **Performance Monitoring:** latency, bandwidth, packet loss, jitter, QoS
 - **Security Analytics:** IDS/IPS, behavioural baselines, ML-based threat detection
 - **Predictive Analytics:** forecast congestion, predict failures, auto-remediate

Xively Cloud Platform for IoT

Xively – IoT Platform as a Service (PaaS) Overview

- **Xively:** Cloud-based IoT Platform-as-a-Service (PaaS) that provides managed infrastructure to connect devices, store sensor data, and build IoT applications without managing servers

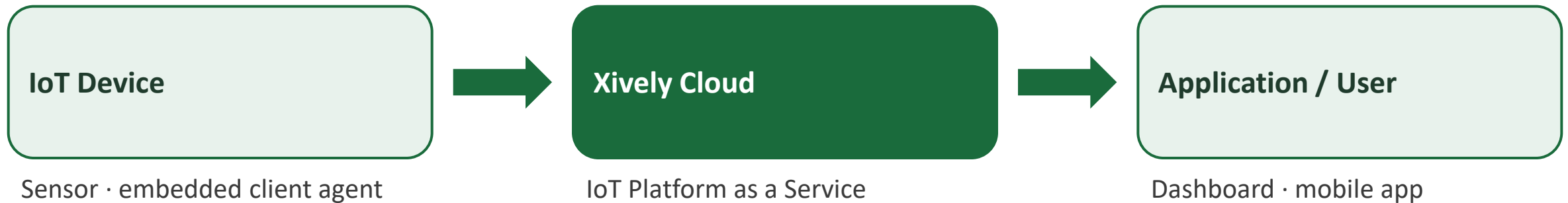
- **History:**

Milestone	Details
2007	Founded as Pachube by Usman Haque in London
2011	Acquired by LogMeIn and renamed Cosm
2013	Rebranded as Xively – launched as a Public Cloud for IoT
2018	Acquired by Google to accelerate Google's IoT development

- **Xively Cloud Services – Key Features**

- **Directory Services** – Manage connected devices and identities
- **Data Services** – Store and retrieve device data streams
- **Trust Engine** – Authenticate and secure connected devices
- **Web-Based Management** – Dashboard for device monitoring and management
- **Messaging** – Uses **MQTT** publish–subscribe protocol
- **API Support** – **REST, WebSockets, and MQTT** interfaces

How Xively Works



- Devices connect via an embedded IoT client agent
- Data streams continuously to the Xively cloud
- Users monitor device status via SDK-based mobile apps (Android / iOS) or the web dashboard
- Alerts and notifications are sent to users when thresholds are crossed

Xively was an early IoT Platform-as-a-Service that provided device connectivity, data management, and application integration – paving the way for modern IoT cloud platforms!

Python Django Web Framework

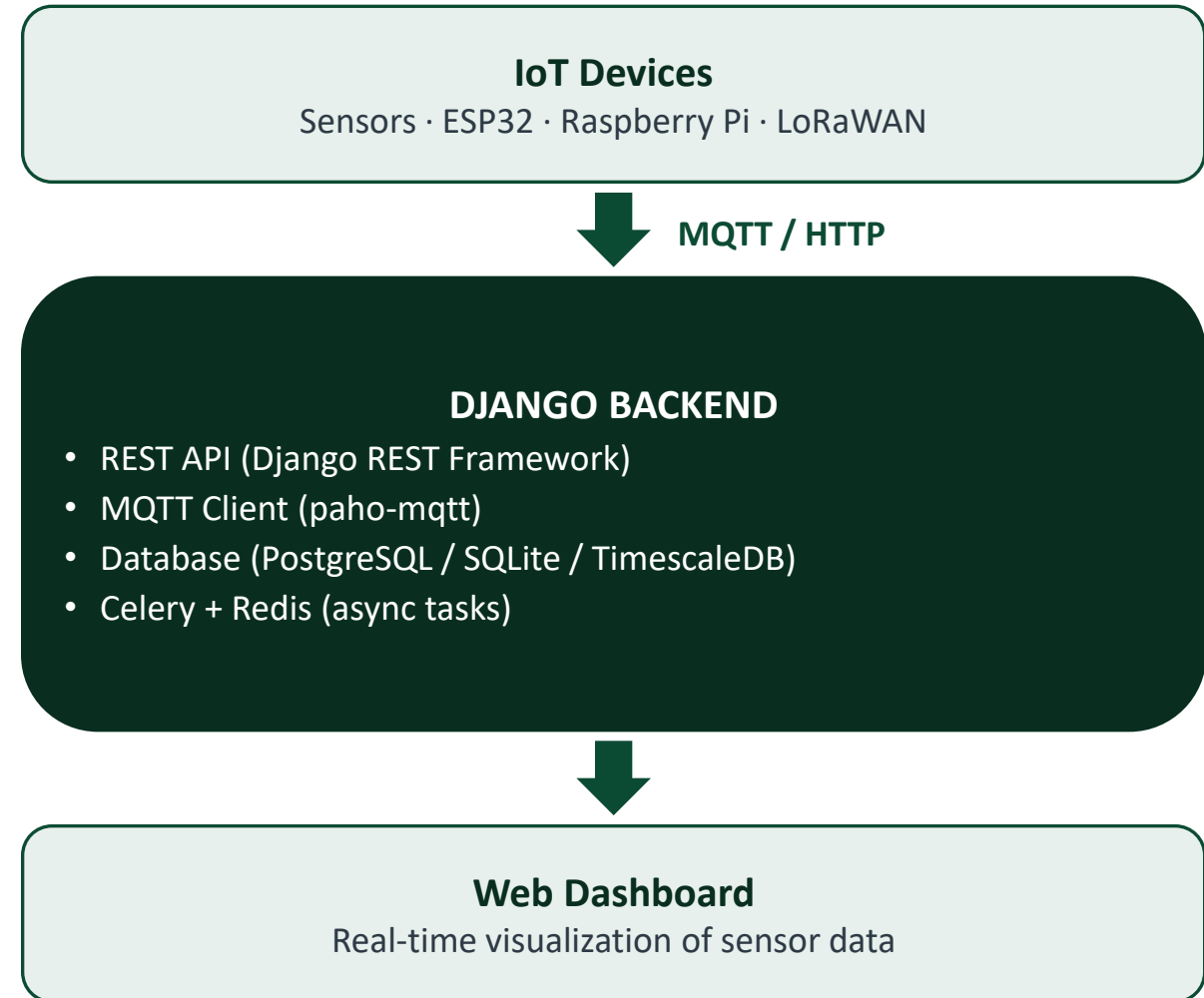
Django – Python Web Framework for IoT Applications

- Django is a high-level Python web framework that enables rapid development of secure and maintainable web applications.
- In IoT, Django is commonly used as the backend framework for building dashboards, APIs, and device management platforms.
- **Why Django for IoT?**
 - **Python-Based** – Built with Python, providing access to a wide range of IoT libraries such as **Paho-MQTT**, **Requests**, and **PySerial** for device communication and integration.
 - **REST Framework** – Enables developers to easily create **RESTful APIs** for communication between IoT devices, applications, and cloud services.
 - **ORM (Object-Relational Mapping)** – Simplifies interaction with databases by allowing **sensor readings, device details, and IoT data** to be stored and retrieved using Python objects.
 - **Admin Panel** – Provides a **built-in web-based administration interface** for managing IoT devices, users, sensor data, and system configurations.
 - **Scalability** – Can be integrated with **Celery and Redis** to handle background tasks, asynchronous processing, and large volumes of IoT data efficiently.
 - **Security** – Includes built-in protection against common web threats such as **SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF)**.

Django MTV Architecture

- **Model** – a Python class that defines your data. Django's ORM turns each class into a database table, so a sensor reading is saved with Python code instead of SQL (e.g. Reading: device, value, time).
- **Template** – the HTML page shown to the user, written with Django Template Language tags such as `{{ value }}` and `{% for %}` that are filled with live sensor data before the page loads.
- **View** – the logic layer: it receives the HTTP request, reads or updates the Model, and returns either a rendered Template or JSON for an IoT device or mobile app.
- **urls.py** – the routing table that links each web address to a View, e.g. `/api/sensor/<device_id>/` returns the latest readings of that device.
- **Key packages** – DRF (REST APIs), Channels + channels-redis (WebSockets for live charts), Celery (background jobs), paho-mqtt / django-mqtt (device messages).

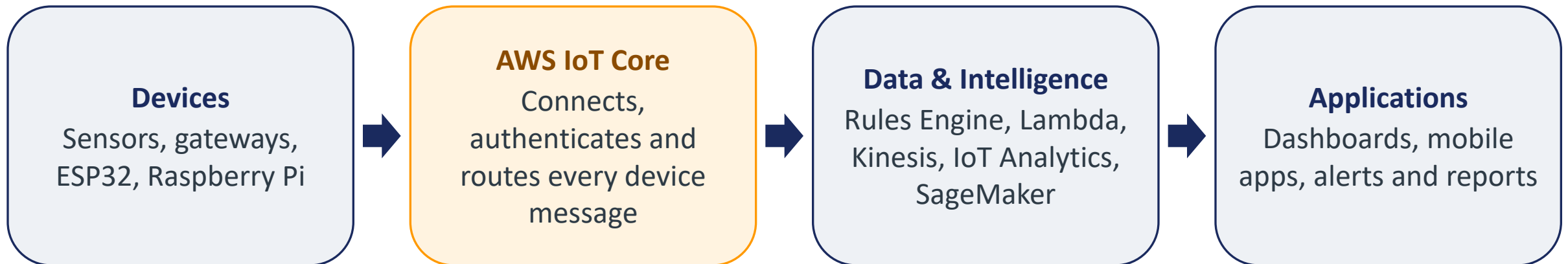
Dr. Mahabub Hasan Mahalat



AWS IoT Platform

AWS IoT Platform – How the Pieces Fit Together

- **AWS IoT is not one product** – it is a family of managed Amazon services that together connect devices, move their data, store and analyse it, and serve applications.
- **Every service has one job.** They are usually grouped into three layers: **connect the device**, process the data, and present the result to the user.
- **AWS IoT Core is the entry point** – all device traffic enters the platform through it, and it decides where each message goes next. That is why we study IoT Core first.



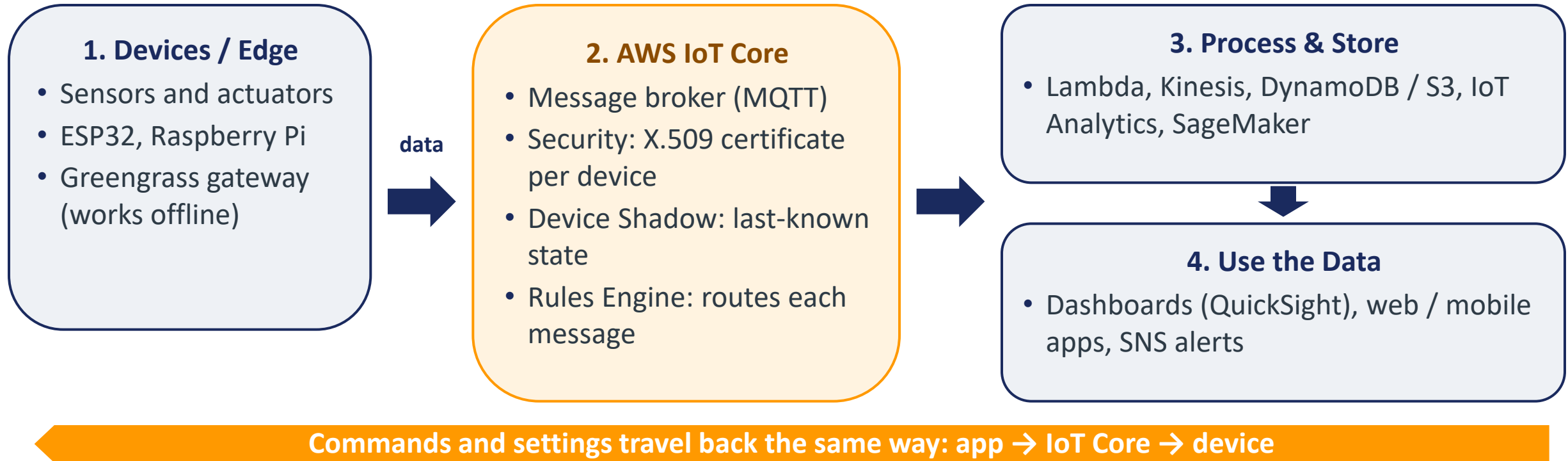
AWS IoT Platform – AWS IoT Core

- **Recall the first layer of the platform:** before any analysis can happen, devices must reach the cloud safely. AWS IoT Core is the service that performs this "connect and route" job, so every other AWS IoT service is fed by it.
- **AWS IoT Core — the central message broker (platform layer 1: connect)**
- **Managed broker** – devices publish and subscribe over MQTT, HTTP or WebSocket. AWS runs the broker, so there is no MQTT server for you to install, patch or scale.
- **Elastic scale** – the same setup works for 10 devices in a lab or billions in production; AWS adds capacity automatically as connections grow.
- **Device Shadow** – a JSON document in the cloud that stores each device's last-known and desired state. An app can read or change a device's state even while the device is asleep or offline; the device syncs when it reconnects.
- **Rules Engine** – SQL-like statements (e.g. SELECT temp FROM 'sensors/+' WHERE temp > 40) that filter incoming messages and forward them to 20+ AWS services such as Lambda, DynamoDB, S3 or SNS — routing without writing server code.
- **Security** – every device gets its own X.509 certificate for TLS authentication plus a JSON IoT policy stating exactly which topics it may publish to or subscribe from.

AWS IoT – Supporting Services

- **Once messages reach IoT Core, these services do the real work:**
- **AWS Greengrass** – runs AWS code (Lambda functions, ML models) on a local gateway such as a Raspberry Pi. The site keeps working when the internet drops and syncs to the cloud later.
- **Amazon Kinesis** – ingests and processes continuous streams of telemetry in real time, so readings can be analysed as they arrive rather than after storage.
- **AWS Lambda** – small serverless functions triggered by an IoT rule (e.g. send an SMS when temperature > 40 °C). You write only the function; AWS scales it automatically.
- **Amazon SageMaker** – trains machine-learning models on collected device data (e.g. predicting motor failure) and, with SageMaker Neo, deploys them back to edge devices.
- **AWS IoT Analytics** – a ready-made pipeline: channel → pipeline → data store → dataset, then visualised as charts in Amazon QuickSight.
- **Fleet Indexing** – search and query millions of device shadows instantly, e.g. list every device whose battery is below 20 %.

AWS IoT – Full Platform Architecture



Devices never talk to the other AWS services directly. They only talk to AWS IoT Core, and its Rules Engine forwards each message to the service that needs it.

Platform Comparison – Xively vs Django vs AWS IoT

Aspect	Xively / Xively-like PaaS	Django Web Framework	AWS IoT Platform
Type	Managed IoT PaaS	Web application framework	Full-stack managed IoT cloud
Primary Role	Device connectivity & data API	IoT web app & REST API server	End-to-end IoT cloud backend
Data Storage	Cloud time-series (Feeds/Datastreams)	PostgreSQL / TimescaleDB	DynamoDB, S3, InfluxDB, Timestream
Protocols	MQTT, HTTP REST, WebSocket	HTTP REST, WebSocket (Channels)	MQTT, HTTP, WebSocket, LoRaWAN
Device Auth	API Key + OAuth2	JWT / Token auth (DRF)	X.509 certificates (per device)
Real-time Data	WebSocket subscriptions	Django Channels + Redis	IoT Core + Kinesis Streams
ML/Analytics	Basic rule triggers	Via external API / Celery tasks	SageMaker, Kinesis Analytics, EMR
Scale	Thousands to millions devices	Depends on infra (EC2/Kubernetes)	Billions of devices (auto-scale)
Cost Model	SaaS subscription	Self-hosted / cloud VM cost	Pay-per-message + service fees
			Enterprise production IoT

IoT Systems Management

with NETCONF and YANG

Need for IoT Systems Management

- **Key Needs for IoT Systems Management**
- **Automating Configuration**
 - Automatically configure IoT devices during deployment.
 - Reduces manual setup and configuration errors.
- **Monitoring Operational & Statistical Data**
 - **Operational Data:** Current device status and performance.
 - **Statistical Data:** collected performance metrics over time, used for analysis and optimization.
 - Continuously monitor device status, performance, and network activity.
 - Helps detect faults and analyze system performance.
- **Improved Reliability**
 - Ensures stable and reliable operation of IoT devices.
 - Enables quick detection and recovery from failures.

Simple Network Management Protocol (SNMP)

- Simple Network Management Protocol (SNMP) is a **widely used network management protocol** that enables administrators to **monitor, manage, and configure network devices**.
- SNMP is commonly used to manage devices such as: Routers, Switches, Servers, Printers, Network gateways
- It helps administrators **monitor network performance, detect faults, and manage network devices remotely**.

Limitations of SNMP

1. Stateless Protocol

- SNMP is **stateless**, meaning each request is processed independently.
- Every request must contain **all required information**.
- The management application must be **intelligent to manage device state**.

2. Connectionless Communication

- SNMP uses **UDP as the transport protocol**.
- UDP is **connectionless and unreliable**.
- Earlier versions had **no guaranteed delivery or acknowledgement of requests**.

3. Limited Configuration Capability

- Many **MIB objects are read-only**.
- Lack of writable objects makes **device configuration difficult using SNMP**.

4. Difficulty in Managing Configuration Data

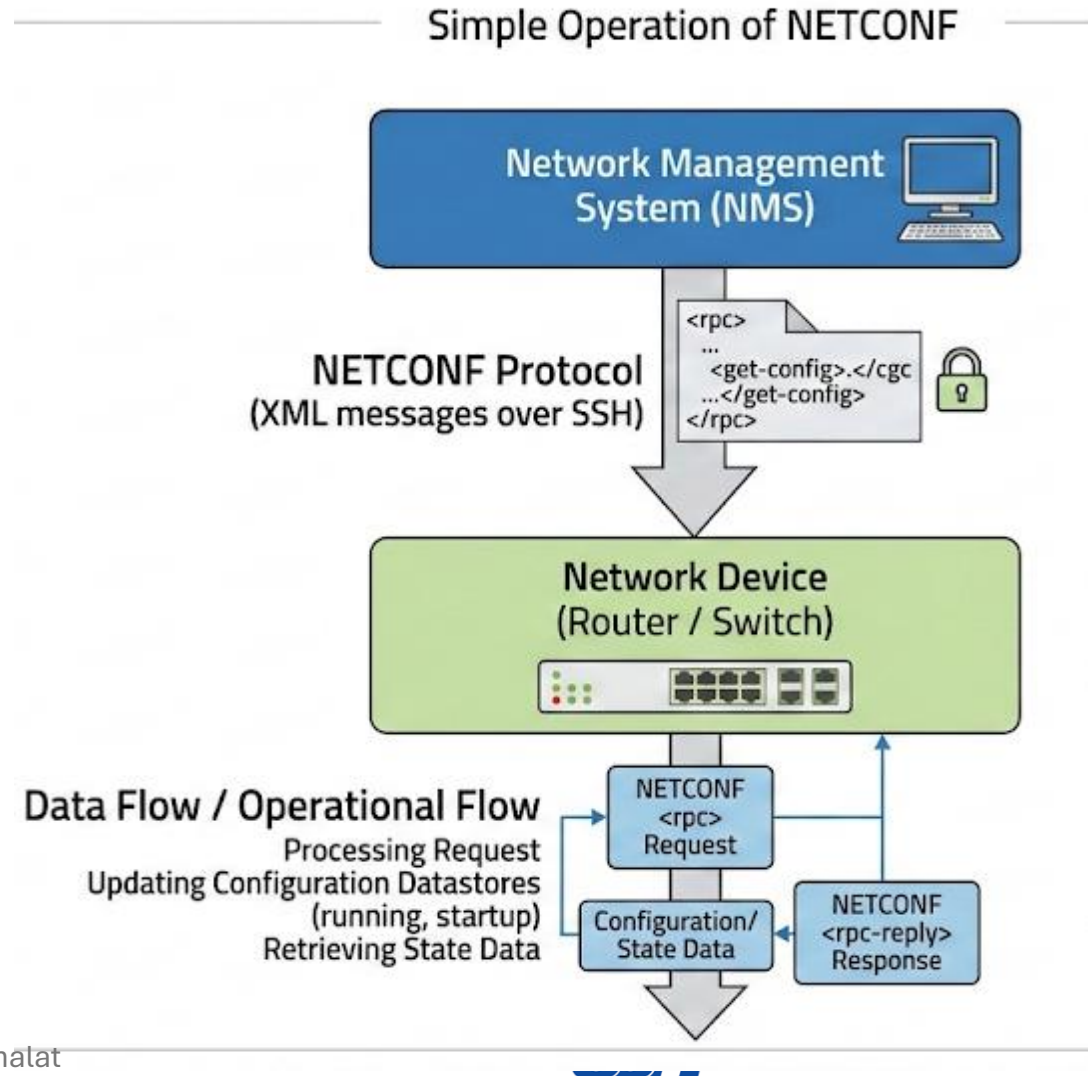
- Hard to distinguish between **configuration data and device state data** in MIBs.
- Retrieving the **complete device configuration** is challenging.

5. Security Limitations

- Early versions (SNMPv1 and SNMPv2) had **weak security mechanisms**.
- Limited support for **authentication and encryption**

Network Configuration Protocol (NETCONF) Introduction

- **NETCONF** is a network management protocol designed to **install, modify, and delete network device configurations**.
- Developed by the Internet Engineering Task Force to improve network configuration management.
- NETCONF **overcomes the limitations of SNMP in network configuration management**.
- **Purpose of NETCONF**
 - NETCONF helps network administrators to:
 - Manage **device configurations efficiently**
 - Automate **network configuration tasks**
 - Maintain **consistent configurations across multiple devices**



NETCONF Overview and Communication

➤ Transport Layer

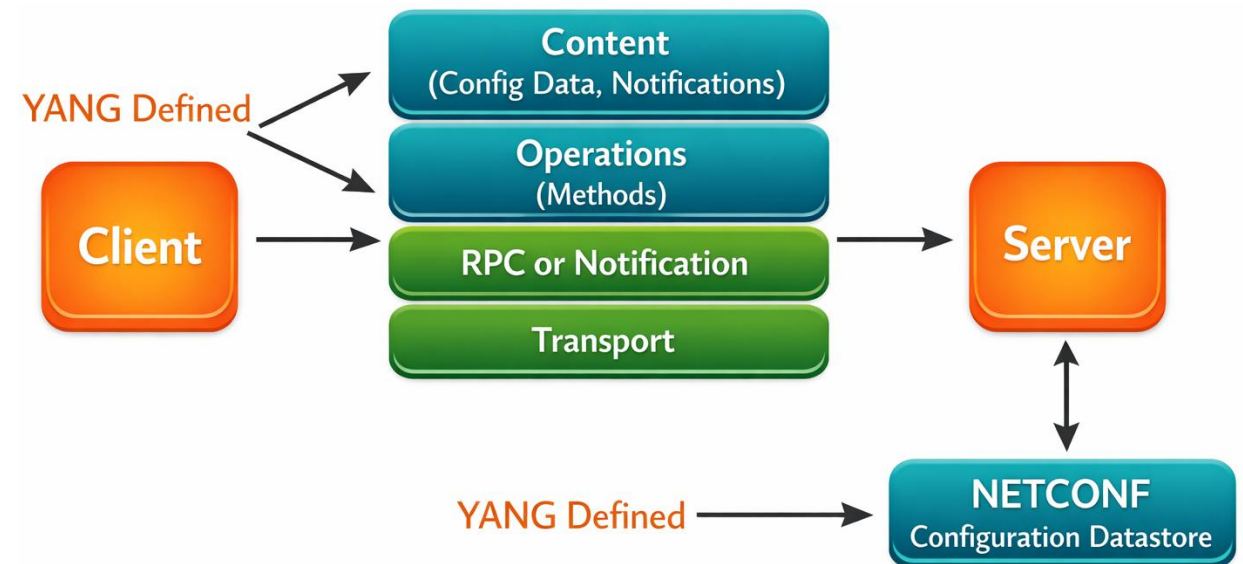
- NETCONF commonly uses **SSH (Secure Shell)** as the transport protocol.
- Provides **secure and reliable communication** between the client and the network device.
- Ensures **end-to-end connectivity and reliable delivery of messages**.

➤ RPC Communication Mechanism

- NETCONF uses **XML-encoded Remote Procedure Calls (RPCs)**.
- RPC messages are used for:
 - **Request messages** from the client
 - **Response messages** from the server
- The RPC layer also supports **notifications from network devices**.

➤ NETCONF Operations

- NETCONF allows administrators to:
- **Retrieve configuration data**
- **Edit or update device configurations**
- **Manage network devices remotely**



NETCONF Data Model and Datastore

❑ Content Layer

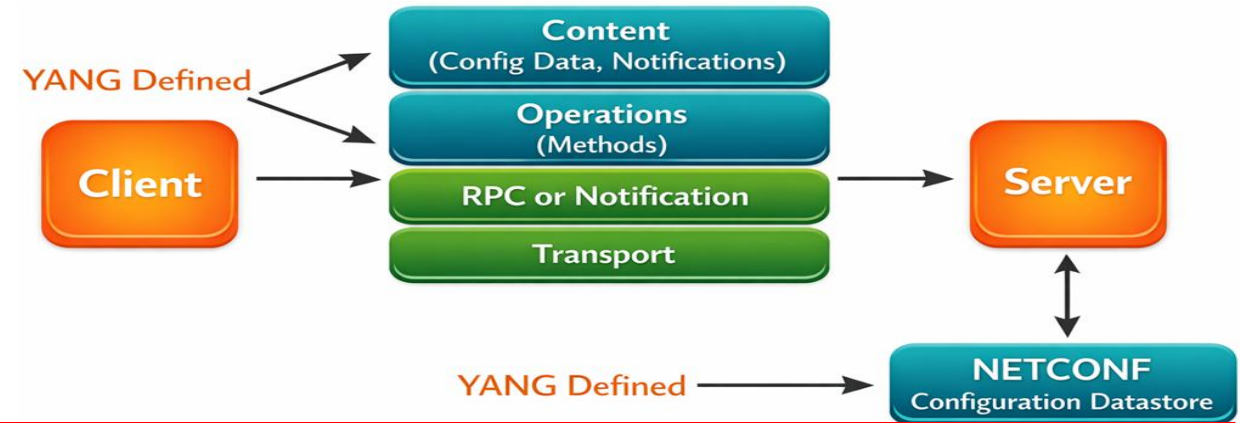
- Contains **configuration data** and **state data** of network devices.
- Data is **encoded in XML format**.

❑ YANG Data Modeling Language

- The structure of configuration and state data is defined using YANG.
- Defines **data models for network configuration**
- Ensures **standardized representation of device configuration data**

❑ Configuration Datastore

- Configuration data is stored in a **NETCONF configuration datastore** on the server.
- Provides **clear separation between configuration data and operational state data**.



Putting it all together

- 1.**Transport:** Secure SSH tunnel opens.
- 2.**RPC:** Client sends an <edit-config> request.
- 3.**Content/YANG:** The request contains XML data (e.g., changing a VLAN ID) that follows **YANG** rules.
- 4.**Datastore:** The Server (Router/Switch) puts that change into the **Candidate** store.
- 5.**Operation:** Client sends a <commit>. The server updates the **Running** store, and the VLAN change goes live.
- 6.**Response:** The Server sends back an <rpc-reply> saying <ok/>.

Introduction to Yang

- **YANG** is a **data modeling language** used to model **configuration data** and **state data** manipulated by the **NETCONF** protocol.
- It defines the **structure and format of data** exchanged between the **NETCONF** client and server.
- A **YANG module** describes:
 - Configuration data
 - State (operational) data
 - RPC operations
 - Notification messages
- YANG acts as the **schema for network configuration data**.

Yang Hierarchical Data Model

- YANG organizes data in a **tree structure**.
- Key elements:
 - **leaf** → single data value
 - **leaf-list** → multiple values
 - **container** → groups related data
 - **list** → repeated data entries

YANG Construct (Node)	Description	Example Use
container	Groups related configuration or state data into a logical structure. It acts like a folder.	A container interface can group interface-related parameters such as name, IP address, and status.
list	Represents a collection of similar entries. Each entry usually has a unique key and can contain multiple child nodes.	A list of network interfaces where each entry represents a different interface (eth0, eth1, etc.).
leaf	Represents a single data value. It cannot contain child nodes.	Interface name, IP address, or device hostname.
leaf-list	Represents multiple values of the same type.	A list of DNS server IP addresses.
config	Indicates whether a node represents configuration data or operational state data .	Interface configuration (config true) vs interface statistics (config false).

YANG Simple Example

```
module network-interface {  
  container interface {  
    leaf name {  
      type string;  
    }  
    leaf ip-address {  
      type string;  
    }  
    leaf status {  
      type string;  
    }  
  }  
}
```

Explanation:

container interface → groups interface information

leaf name → interface name (e.g., eth0)

leaf ip-address → IP address of the interface

leaf status → operational state (up/down)

Important Capabilities of YANG

- Defines **clear data structure for network configuration**
- Supports **constraints on data values**
- Allows **importing definitions from other modules**
- Can model both:
 - **Configuration data**
 - **Operational (state) data**

IoT Systems Management using NETCONF–YANG

- IoT systems management requires **automated configuration, monitoring, and control of large numbers of devices**.
- **NETCONF and YANG** provide structured and programmable mechanisms for managing network and IoT devices.
- **Management Layer Components**
 - These components interact with administrators and management applications.

Component	Description
Management System	Central platform responsible for monitoring and managing IoT devices and network infrastructure.
Management API	Interface through which applications or administrators interact with the management system to perform configuration and monitoring operations.
Transaction Manager	Ensures configuration changes are applied atomically and consistently across devices. Handles commit and abort operations.
Rollback Manager	Restores the previous configuration if a configuration update fails, ensuring system reliability and fault recovery.

IoT Systems Management using NETCONF–YANG...

➤ Data Modeling & Validation Components

- These components ensure that configuration data is **structured and valid** before being applied to devices.

Component	Description
Data Model Manager	Manages YANG data models that define the structure of configuration and operational data in IoT devices.
Configuration Validator	Verifies configuration data against the YANG schema to ensure correctness before applying it to the device.
Configuration Database	Stores device configuration data and system state for management and future reference.

IoT Systems Management using NETCONF–YANG...

➤ Configuration & Data Interface Components

➤ These components provide interfaces for accessing configuration and operational data.

Component	Description
Configuration API	Allows applications to retrieve, modify, and update configuration information stored in the configuration database.
Data Provider API	Interface used by devices or services to supply operational data (status, statistics, performance metrics) to the management system.

UIT2734 · IOT ARCHITECTURES AND APPLICATIONS

End of Unit IV

Data Analytics and Supporting Services

Thank you!